Fast Fourier Transform (FFT)

Derivation of Core Equations

Ira A. Fulton Schools of Engineering
Arizona State University

Computation of the Discrete Fourier Transform (DFT)

Recall DFT:

- $x[n] \xleftrightarrow{N-point\ DFT} X[k]$
- $X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn}, k = 0, 1, \ldots, N-1$

Let $e^{-j2\pi\frac{kn}{N}} \triangleq W_N^{kn}$ (twiddle factor)

- $W = e^{-j2\pi}$
- $X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, k = 0, 1, \ldots, N-1$

Direct Computation of DFT: Computation Complexity

DFT: $X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, k = 0, 1, \ldots, N-1$

Assumption: let $W_N^{kn} = e^{-j\frac{2\pi}{N}kn}, k = 0, \ldots, N-1$ **be precomputed and stored in a table.**

To compute one DFT sample, we need

- N complex multiplications (CMULTs)
- N-1 complex additions (CADDs)

Total computations for N-point DFT:

- N^2 complex multiplications (CMULTs)
- N(N-1) complex additions (CADDs)

Example

- **Consider 5 minutes of speech sampled at 8KHz:**
- **Length of discrete domain samples x[n] is**
  - $N = 5 * 60 * 8000 = 2.4 * 10^6$ samples
- **Number of computations (simplified):**
  - N-point DFT: N^2 CMULTs = $4N^2$ MULTs = $4 * 2.4^2 * 10^{12}$ MULTs
  - Note: 1 CMULT = 4 MULTs + 2 ADDs (not counting ADDs)
- **DSP at 168MHz with 1 instruction per clock cycle, i.e., 168MIPS**
- **Time to compute N-point DFT for 5 minutes of speech:**
  - $4 * 2.4^2 * 10^{12} / (168 * 10^6) \approx 137143$ sec ≈ 38 hr

Question

For a N = 64-point DFT, how many real multiplication operations are needed if it is implemented directly using the DFT equation?

a) 448

b) 768

c) 4096

d) 16384

Fast Fourier Transform (FFT)

This is NOT a new transform.

It is just the DFT X[k] but computed using a faster way.

Requirement:

- Size of DFT is restricted to be a power of some integer $N = R^{\alpha}$.
- Most commonly R = 2 (radix 2), i.e., $N = 2^{\alpha}$.

Fast Fourier Transform (FFT)

Rationale: arrange computations into smaller transformations.

Two methods:

- Decimation in Time (DIT)
  - Decompose x[n] into smaller subsequences
- Decimation in Frequency (DIF):
  - Decompose X[k] into smaller subsequences

Our focus: Decimation in Time algorithm

Decimation In Time FFT Algorithm

- **Break x[n] down into even and odd subsequences:**

$$X[k] = \sum_{n=0}^{N-1} x[n]e^{-j\frac{2\pi}{N}kn}, k = 0, 1, \dots, N-1$$

$$= \sum_{n=2m} x[n]e^{-j\frac{2\pi}{N}kn} + \sum_{n=2m+1} x[n]e^{-j\frac{2\pi}{N}kn}$$

- **When n is even:**

$$0 \le n \le N-1 \Rightarrow 0 \le 2m \le N-1$$

$$\Rightarrow 0 \le m \le \frac{N-1}{2} \Rightarrow 0 \le m < \frac{N-1}{2} \Rightarrow 0 \le m \le \frac{N-2}{2} = \frac{N}{2} - 1 \Rightarrow 0 \le m \le \frac{N}{2} - 1$$

- **When n is odd:**

$$0 \le n \le N-1 \Rightarrow 0 \le 2m+1 \le N-1$$

$$\Rightarrow 0 \le m \le \frac{N-2}{2} \Rightarrow 0 \le m \le \frac{N}{2} - 1$$

Decimation In Time FFT Algorithm

$$X[k] = \sum_{n=2m} x[n]e^{-j\frac{2\pi}{N}kn} + \sum_{n=2m+1} x[n]e^{-j\frac{2\pi}{N}kn}$$

$$= \sum_{m=0}^{\frac{N}{2}-1} x[2m]e^{-j\left(\frac{2\pi}{N}k\right)2m} + \sum_{m=0}^{\frac{N}{2}-1} x[2m+1]e^{-j\left(\frac{2\pi}{N}k\right)(2m+1)}$$

- **Let** $x_e[m] = x[2m]$ **and** $x_o[m] = x[2m+1]$**.**

- **Note:** $e^{-j\left(\frac{2\pi}{N}k\right)(2m+1)} = e^{-j\left(\frac{2\pi}{N}k\right)2m} e^{-j\left(\frac{2\pi}{N}k\right)} = e^{-j\left(\frac{2\pi}{N}k\right)2m} W_N^k$

$$X[k] = \sum_{m=0}^{\frac{N}{2}-1} x_e[m]e^{-j\left(\frac{2\pi}{N}k\right)2m} + W_N^k \sum_{m=0}^{\frac{N}{2}-1} x_o[m]e^{-j\left(\frac{2\pi}{N}k\right)2m}$$

$$= \sum_{m=0}^{\frac{N}{2}-1} x_e[m]e^{-j\left(\frac{\frac{2\pi}{N}k}{\frac{N}{2}}\right)m} + W_N^k \sum_{m=0}^{\frac{N}{2}-1} x_o[m]e^{-j\left(\frac{\frac{2\pi}{N}k}{\frac{N}{2}}\right)m}$$

Decimation In Time FFT Algorithm

- **Break N-point DFT down into two N/2-point DFTs:**

$$X[k] = \sum_{m=0}^{\frac{N}{2}-1} x_e[m] e^{-j\left(\frac{2\pi}{\frac{N}{2}}k\right)m} + W_N^k \sum_{m=0}^{\frac{N}{2}-1} x_o[m] e^{-j\left(\frac{2\pi}{\frac{N}{2}}k\right)m}$$

$$= X_e[k] + W_N^k X_o[k], k = 0, 1, \ldots, \frac{N}{2} - 1$$

- **where**

$$X_e[k] = \sum_{m=0}^{\frac{N}{2}-1} x_e[m] e^{-j\left(\frac{2\pi}{\frac{N}{2}}k\right)m},$$

$$X_o[k] = \sum_{m=0}^{\frac{N}{2}-1} x_o[m] e^{-j\left(\frac{2\pi}{\frac{N}{2}}k\right)m}.$$

- **But this is only half of the values, we also need k = N/2, …, N-1.**

Decimation In Time FFT Algorithm

First some facts:

- $X_e[k]$ and $X_o[k]$: periodic with period N/2.
  - $X_e\left[k+\frac{N}{2}\right] = \sum_{m=0}^{\frac{N}{2}-1} x_e[m] e^{-j\left(\frac{2\pi}{\frac{N}{2}}(k+\frac{N}{2})\right)m} = X_e[k]$
  - $X_o\left[k+\frac{N}{2}\right] = \sum_{m=0}^{\frac{N}{2}-1} x_o[m] e^{-j\left(\frac{2\pi}{\frac{N}{2}}(k+\frac{N}{2})\right)m} = X_o[k]$
- $W_N^{k+\frac{N}{2}} = e^{-j\frac{2\pi}{N}(k+\frac{N}{2})} = -e^{-j\frac{2\pi}{N}k} = -W_N^k$

Decimation In Time FFT Algorithm

- **We already have** $X[k] = X_e[k] + W_N^k X_o[k], k = 0, 1, \dots, \frac{N}{2} - 1$
- **We also need the values of** $X[k], k = \frac{N}{2}, \frac{N}{2} + 1, \dots, N - 1$
- **From last page's facts, we get**

$$X\left[k + \frac{N}{2}\right] = X_e\left[k + \frac{N}{2}\right] + W_N^{k+\frac{N}{2}} X_o\left[k + \frac{N}{2}\right]$$

$$= X_e[k] - W_N^k X_o[k], k = 0, 1, \dots, \frac{N}{2} - 1$$

- **This is the other half we need.**

FFT Core Equations

- A N-point FFT is decomposed into two N/2-point FFTs:

$$X[k] = X_e[k] + W_N^k\, X_o[k], k = 0, 1, \dots, \frac{N}{2} - 1$$

$$X\left[k + \frac{N}{2}\right] = X_e[k] - W_N^k\, X_o[k], k = 0, 1, \dots, \frac{N}{2} - 1$$

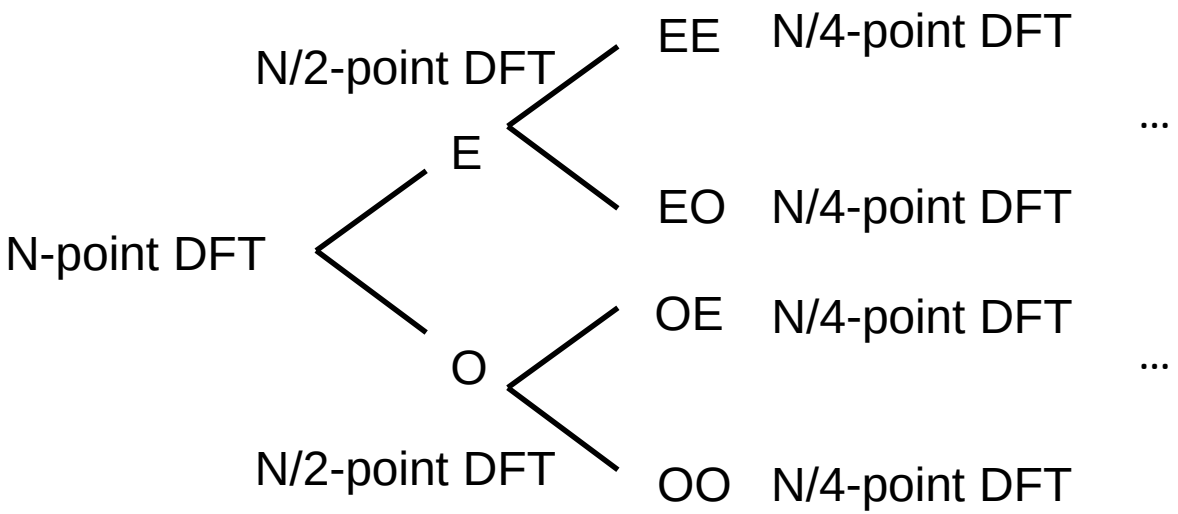
$$X_e[k] = \sum_{m=0}^{\frac{N}{2}-1} x_e[m] e^{-j\left(\frac{2\pi}{\frac{N}{2}}k\right)m}$$

$$X_o[k] = \sum_{m=0}^{\frac{N}{2}-1} x_o[m] e^{-j\left(\frac{2\pi}{\frac{N}{2}}k\right)m}$$

$$W_N^k = e^{-j\frac{2\pi}{N}k}$$

Decimation In Time FFT Algorithm

- **Continue to break down each sum into even and odd terms until only one value is left in the summation.**
- **Number of stages needed: $\log_2 N$**



Ira A. Fulton Schools of Engineering

Arizona State University